

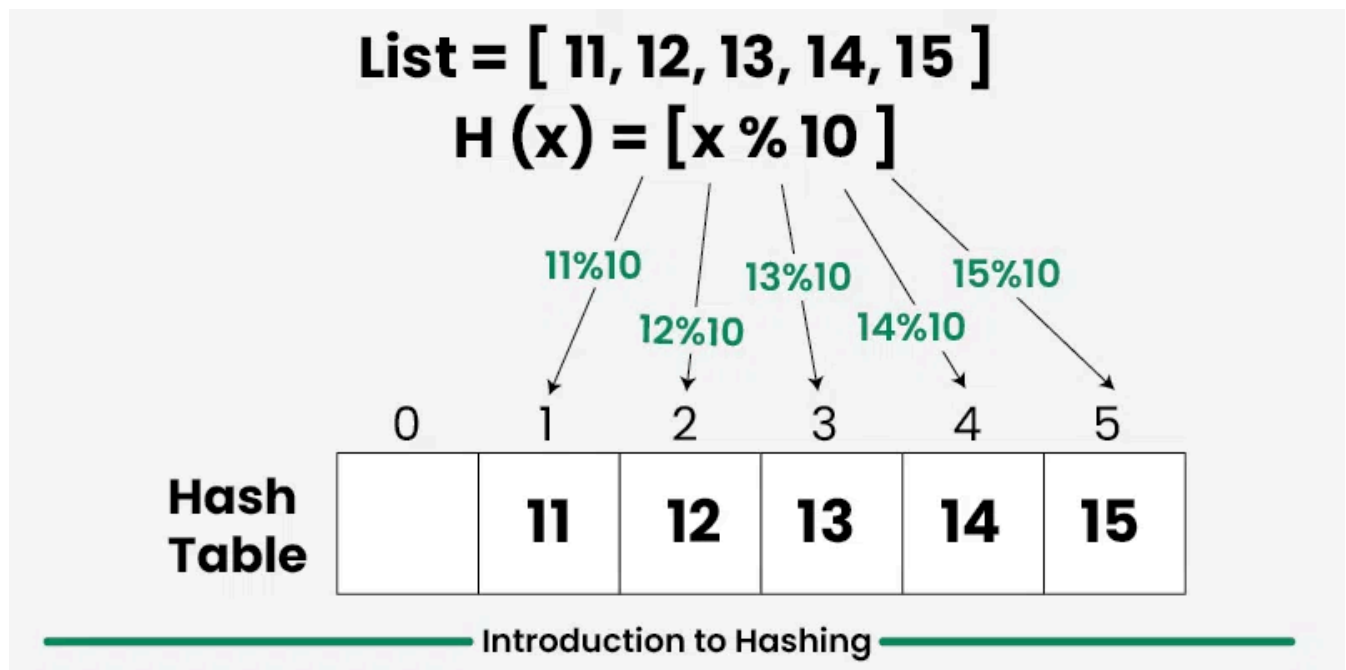
hash_functions

Subject: #ands #hash_functions

Topic: Hash functions

Hashing

Hashing refers to the process of generating a small sized output (which can usually be used as an index in a table) from an input of typically large variable size. Hashing uses mathematical formulas known as hash functions to do the transformation. This technique determines the index or location for the storage of an item in the **Hash Table**.



Average time:

It supports search, insert and delete in $O(1)$ in average, which is more efficient than other popular data structures. (Examples to be placed)

Usages

It is used in dictionaries, frequency counting, and maintaining data for quick access by key.

Warning

Hash is not used in situations where:

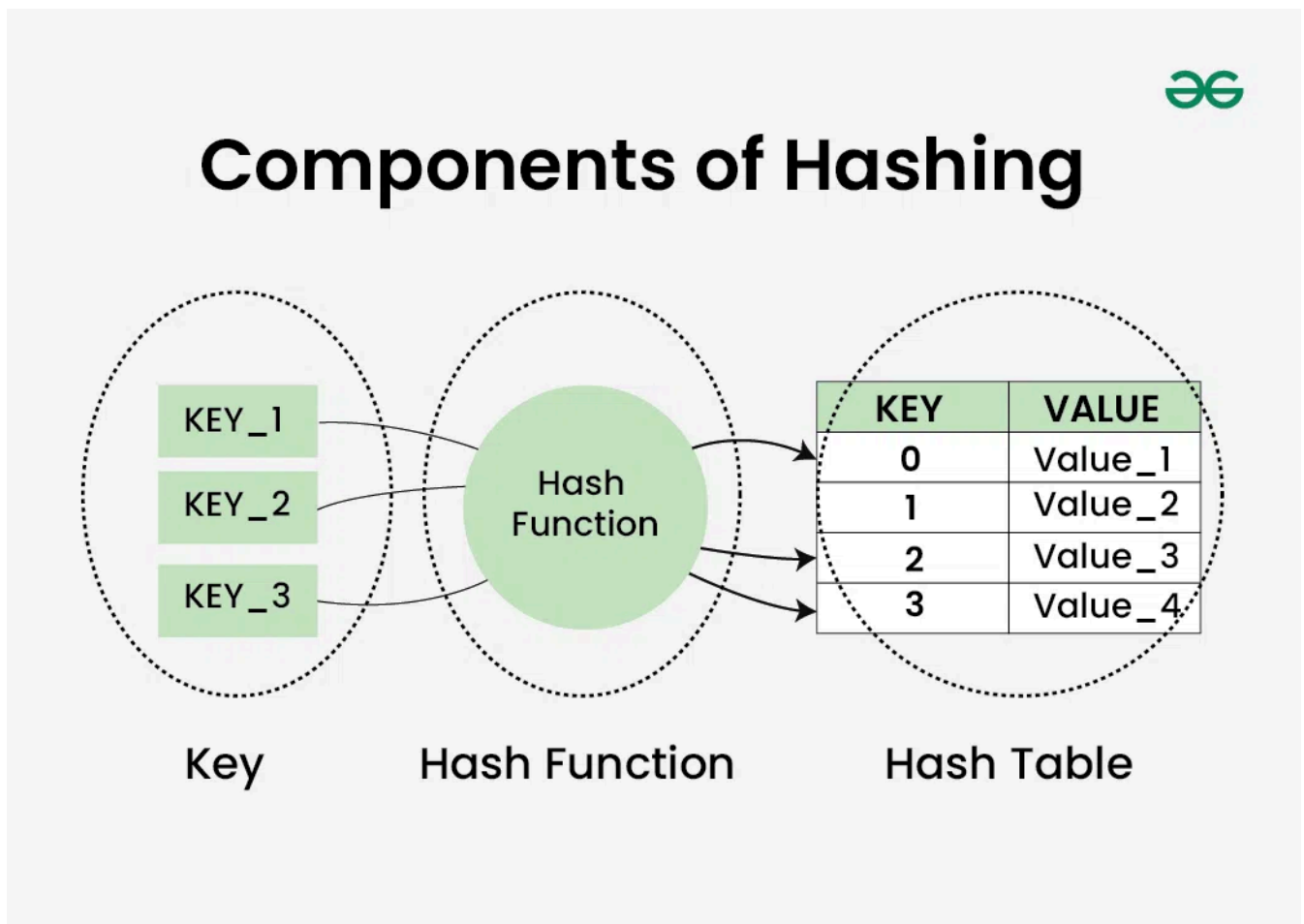
1. There is a need to maintain sorted data along with search, insert and delete.
2. When strings are key and we need operations like prefix search along with search, insert and delete.
3. When there is a need to use operations like floor and ceiling along with search, insert and/or delete.

Types

1. Hash Set: A collection of unique keys.
2. Hash Map: Collection of key value pairs with keys being unique.

Components

1. Key: A Key can be anything string or integer which is fed as input in the hash function the technique that determines an index or location for storage of an item in a data structure.
2. Hash function: Receives the input key and returns the index of an element in an array called a hash table. The index is known as the hash index .
3. Hash table: Hash table is typically an array of lists. It stores values corresponding to the keys. Hash stores the data in an associative manner in an array where each data value has its own unique index.



Implementation

Suppose that we have a set of strings {"ab", "cd", "efg"}

1. We know that hash functions are used to calculate the hash value which acts as the index of the data structure where the value will be stored.
2. We assign numbers to all alphabetical characters. (a = 1, b = 2)
3. Now we are able to have the sum of all the characters of strings.
4. We assume we have a table of 7, to store these strings. The hash function that is used here is the sum of the characters in key mod Table size. We can compute the location of the string in the array by taking the `sum(string) mod 7`.
5. We would be able to store: - "ab" in $3 \bmod 7 = 3$, "cd" in $7 \bmod 7 = 0$, and "efg" in $18 \bmod 7 = 4$.

Mapping Key with indices of Array

0	1	2	3	4	5	6
cd			ab	egf		

```

1
2  long long compute_hash(string const& s) {
3      const int p = 31;
4      const int m = 1e9 + 9;
5      long long hash_value = 0;
6      long long p_pow = 1;
7      for (char c : s) {
8          hash_value = (hash_value + (c - 'a' + 1) * p_pow) % m;
9          p_pow = (p_pow * p) % m;
10     }
11     return hash_value;
12 }
13

```

Hash functions

A hash function creates a mapping from an input key to an index in hash table.

Key properties of hash functions:

- Deterministic. It must produce the same output for the same input.
- Fixed Output Size. The output of a hash functions should have a fixed size, regardless of the size of the input.
- Efficient.
- Uniformity. The hash function must distribute the hash values uniformly across the output space to avoid clustering.
- Pre-image resistance: It should be computationally infeasible to reverse it.
- Collision Resistance.

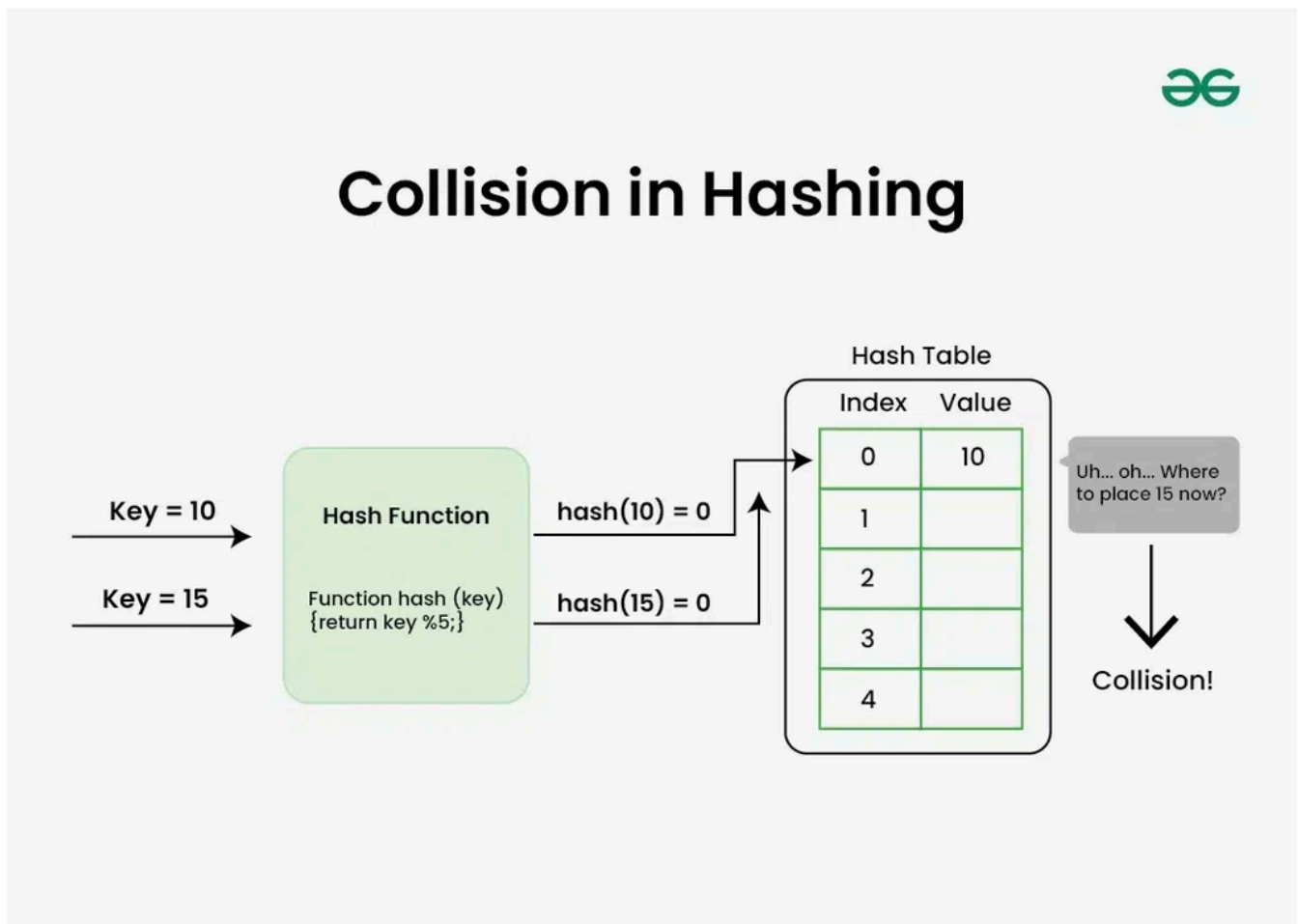
Types of Hash functions:

- Division Method, $h(k) = k \bmod m$ Where k is the key and m is a prime number.
- Multiplication Method, $h(k) = \lfloor m(kA \bmod 1) \rfloor$ Where $\lfloor \cdot \rfloor$ denotes the floor function.
- Mid-Square Method, in the mid-square method, the key is squared, and the middle digits of the result are taken as the hash value.
- Folding Method, the folding method involves dividing the key into equal parts, summing the parts, and then taking the modulo with respect to m .
- Universal Hashing, $h(k) = ((a \cdot k + b) \bmod p) \bmod m$ Where a and b are randomly chosen constants, p is a prime number greater than m , and k is the key.

Collusions:

When two or more keys have the same hash value, a **collision** happens.

For the example above: {"ab", "ba"} both have the same hash value, and string {"cd", "be"} also generate the same hash value. This is known as **collision** and it creates problem in searching, insertion, deletion, and updating of value.



The probability of a hash collision depends on the size of the algorithm, the distribution of hash values and the efficiency of Hash function.

Collision Resolution

Separate Chaining

The point is to make each cell of the hash table point to a linked list of records that have the same hash function value. Chaining is simple but requires additional memory outside the table.

Example:

Hash function = $\text{key} \% 5$,

Elements = 12, 15, 22, 25 and 37.

Open Addressing

All elements are stored in the hash table itself. Each table entry contains either a record or NIL (the key of the hash table). When searching for an element, we examine the table slots one by one until the desired element is found or it is clear that the element is not in the table.

Linear Probing.

In linear probing, the hash table is searched sequentially that starts from the original

location of the hash. If in case the location that we get is already occupied, then we check for the next location.

Algorithm:

Calculate the hash key. i.e. **key = data % size**

Check, if **hashTable[key]** is empty store the value directly by **hashTable[key] = data**

If the hash index already has some value then check for next index using **key = (key+1) % size**

Check, if the next index is available hashTable[key] then store the value. Otherwise try for next index.

Do the above process till we find the space.

Example: <https://www.geeksforgeeks.org/dsa/implementing-hash-table-open-addressing-linear-probing-cpp/>

Quadratic Probing

Quadratic probing operates by taking the original hash index and adding successive values of an arbitrary quadratic polynomial until an open slot is found.

This method is also known as the mid-square method because in this method we look for i^2 -th probe (slot) in i -th iteration and the value of $i = 0, 1, \dots, n-1$. We always start from the original hash location. If only the location is occupied then we check the other slots.

Let hash(x) be the slot index computed using the hash function and n be the size of the hash table.

If the slot hash(x) % n is full, then we try (hash(x) + 1²) % n.

If (hash(x) + 1²) % n is also full, then we try (hash(x) + 2²) % n.

If (hash(x) + 2²) % n is also full, then we try (hash(x) + 3²) % n.

This process will be repeated for all the values of **i** until an empty slot is found

Example: <https://www.geeksforgeeks.org/dsa/quadratic-probing-in-hashing/>

Double Hashing

Double hashing make use of two hash function;

- The first hash function is **h1(k)** which takes the key and gives out a location on the hash table. But if the new location is not occupied or empty then we can easily place our key.
- But in case the location is occupied (collision) we will use secondary hash-function **h2(k)** in combination with the first hash-function **h1(k)** to find the new location on the hash table.

This combination of hash functions is of the form:

$$h(k, i) = (h_1(k) + i \cdot h_2(k))$$

where

- i is a non-negative integer that indicates a collision number,
- k = element/key which is being hashed
- n = hash table size.

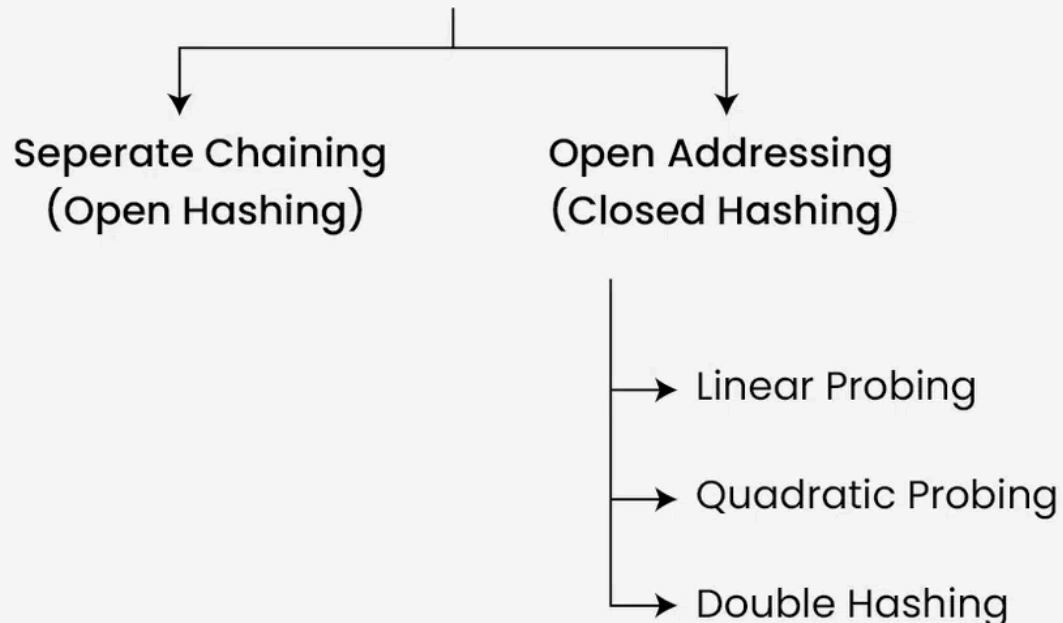
Note

Time complexity: $O(n)$

Example: <https://www.geeksforgeeks.org/dsa/double-hashing/>



Collision Resolution Techniques



Sources :

https://disk.yandex.ru/d/IjK_JW7VCCRMPg/%D0%90%D0%B8%D0%A1%D0%942/%D0%BB%D0%B5%D0%BA%D1%86%D0%B8%D1%8F/%D0%9A%D1%83%D1%80%D0%B5%D0%BD%D0%BA%D0%BE%D0%B2%20%D0%92.%D0%92.%202024-09-07T11-36-32Z.mp4

<https://www.geeksforgeeks.org/dsa/hashing-data-structure/>

<https://cp-algorithms.com/string/string-hashing.html>

Additional materials :

<https://www.cs.usfca.edu/~galles/visualization/OpenHash.html>

Review questions :

1. What is hashing?

2. What is the average time complexity of search, insert, and delete in hashing?

3. Name three situations where hashing is not suitable.

[Redacted]

4. What is a Hash Set?

[Redacted]

5. What is a Hash Map?

[Redacted]

6. What are the three main components of hashing?

[Redacted]

7. What does a hash function do?

[Redacted]

8. List the key properties of a good hash function.

[Redacted]

9. What is the division method of hashing?

[Redacted]

10. What is the multiplication method of hashing?

[Redacted]

11. What is the mid-square method of hashing?

[Redacted]

12. What is the folding method of hashing?

[Redacted]

13. What is universal hashing?

[Redacted]

14. What is a collision in hashing?

[Redacted]

15. Name two common collision resolution strategies.

[Redacted]

16. How does separate chaining work?

[Redacted]

17. How does open addressing work?

[Redacted]

18. What is the difference between separate and open addressing?

[Redacted]

19. What is linear probing?

[Redacted]

20. What is quadratic probing?

21. What is double hashing?

22. What is the time complexity of double hashing?

Seminar

This a binary way of removing a степень in a non recursive method:

log (k)

```
1  ll po(ll x, ll k, ll mod)
2  {
3      ll res = 1;
4      while (k > 0)
5      {
6          if (k % 2 == 1)
7          {
8              res = (res * x) % mod;
9              k--; //this action can be removed because of the way cpp code works
10         }
11         x = (x * x) % mod; //here it is important to notice that we've saved the operation
12         k /= 2;
13     }
14     return res;
15 }
16
```

Now the core parts of the algorithm:

```
1
2  int n;
3  string s;
4  vector<ll> h(n + 1, 0)
5  xx(n + 1, 1) //(here we save the x^i)
6  xx[1] = x
7  ix(n + 1, 1) //(here we save the x^{-1})
8  ix[1] = po(x, mod - 2, mod)
9  ll x = 257
10 mod = 1e9 + 7
11
```

the implementation:


```

1
2  for (int i = 2; i < n + 1; i++)
3  {
4      xx[i] = xx[i - 1] * xx[1] % mod;
5      ix[i] = ix[1] * xx[i] % mod;
6  }
7
8  for (int i = 1; i < n + 1; i++)
9  {
10     h[i] = (h[i - 1] + (s[i - 1] - 'a' + 1) * xx[i - 1]) % mod;
11 }
12
13

```

this formula is usually used a separate index of the hashes

```

1  ll hash(int l, int r)
2  {
3      return (h[r + 1] + mod - h[l]) * ix[l] % mod;
4  }
5

```